

Day 04: Network Penetration Testing & Traffic Analysis Report

Prepared by: Yash Jayendrakumar Patel

Project: Cybersecurity Internship / Virtual Lab Task

Date: 27-Aug-2025

Version: 1.0

1. Executive Summary

This report documents the activities performed during Day 04 of the cybersecurity virtual lab, including network scanning, vulnerability enumeration, penetration testing, and traffic analysis. The tasks were performed on a controlled lab environment using Kali Linux (attacker) and Ubuntu Linux (target) VMs connected over a bridged network.

The report is structured according to NIST and ISO 27001 standards, ensuring proper documentation of scope, methodology, findings, and analysis for organizational acceptance.

Objectives:

- Perform network enumeration and scanning using Nmap and Metasploit.
- Identify services and potential vulnerabilities.
- Capture and analyze network traffic using Python and Wireshark.
- Document findings and insights in a professional standard-compliant report.

2. Scope

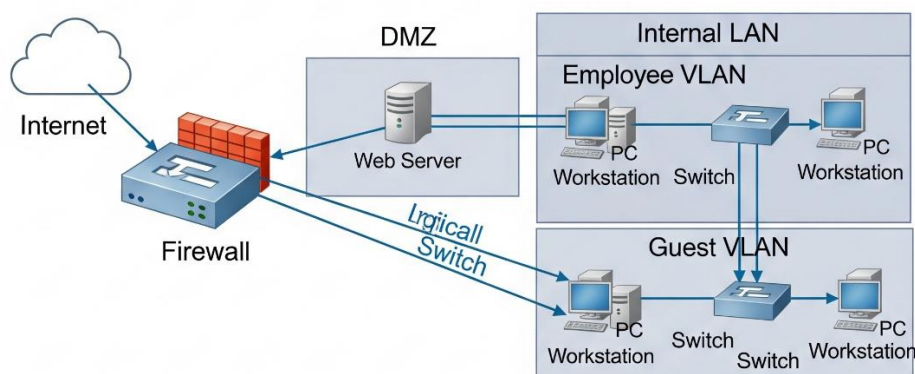
The scope of this task includes:

- Kali Linux VM (192.168.0.102) acting as the attacker.
- Ubuntu Linux VM (192.168.0.101) acting as the target.
- Network services including SSH and HTTP on Ubuntu VM.
- Penetration testing limited to scanning and safe exploit enumeration.
- Traffic analysis focused on ICMP, TCP, and HTTPS packets.

Exclusions:

- Any attacks on external networks.
- Exploits that could harm the target system.

4. Network Topology



Topology Explanation:

- Both VMs are connected via a **Bridged Adapter**, placing them on the same subnet for direct communication.
 - Kali Linux (attacker) can communicate with Ubuntu Linux (target) using ICMP, TCP, and HTTP.
 - Network addresses:
 - Kali Linux: 192.168.0.102
 - Ubuntu Linux: 192.168.31.208 / 192.168.0.101
 - **Rationale:** The bridged network setup allows realistic traffic capture and service enumeration without NAT interference.
-

5. Penetration Testing with Metasploit

5.1 Methodology:

- Identified live hosts using ping and Nmap scans.
- Performed service enumeration with Nmap (-sV) and Metasploit HTTP auxiliary modules.
- Evaluated potential exploits available for detected services.

5.2 Commands Used:

```
nmap -sV 192.168.31.208
```

```
nmap -sV -p 22 192.168.31.208
```

```
nmap -sV -p 80,443 192.168.31.208
```

```
msfconsole
```

```
use auxiliary/scanner/http/http_version
```

```
set RHOSTS 192.168.31.208
```

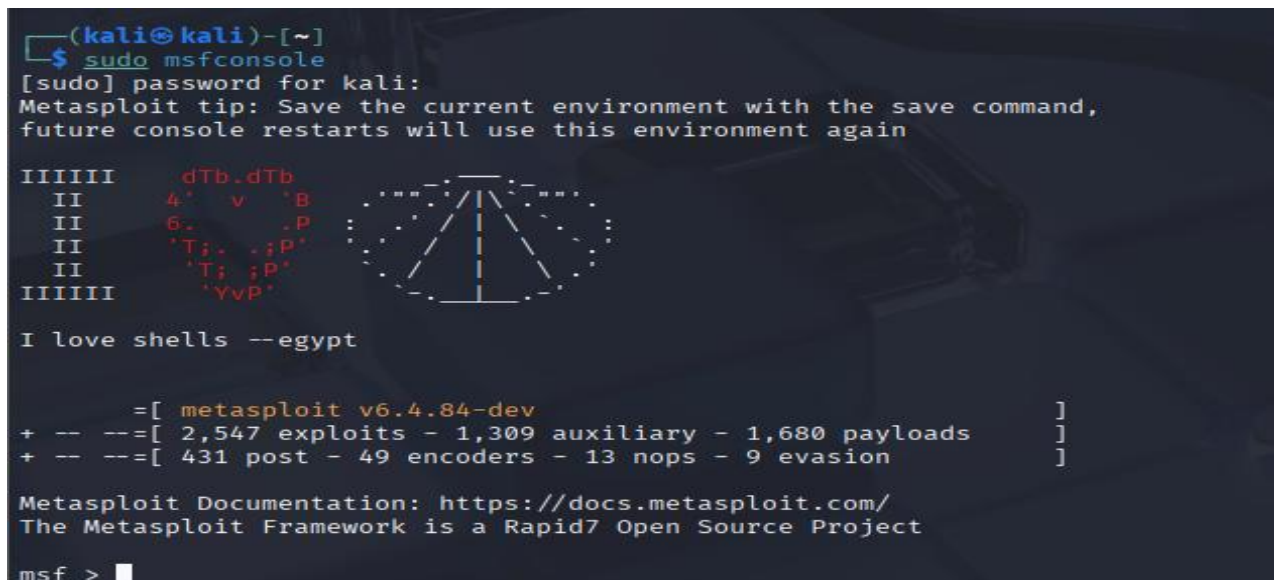
```
run
```

```
search type:exploit name:apache
```

5.3 Results:

- **SSH (Port 22):** Open, OpenSSH 8.9p1 Ubuntu 3ubuntu0.13
- **HTTP (Port 80):** Open, Apache 2.4.52 (Ubuntu)
- **HTTPS (Port 443):** Closed

Screenshot Placement:



```
(kali@kali)-[~]
$ sudo msfconsole
[sudo] password for kali:
Metasploit tip: Save the current environment with the save command,
future console restarts will use this environment again

IIIIII  dTb.dTb
II      4'  v  'B
II      6.   .P
II      'T; .;P'
II      'T; ;P'
II      'YvP'
IIIIII

I love shells --egypt

      =[ metasploit v6.4.84-dev ]
+ -- --=[ 2,547 exploits - 1,309 auxiliary - 1,680 payloads ]
+ -- --=[ 431 post - 49 encoders - 13 nops - 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > 
```

```
msf auxiliary(scanner/ssh/ssh_login) > set USERNAME root
USERNAME => root
msf auxiliary(scanner/ssh/ssh_login) > unset USER_FILE
Unsetting USER_FILE...
msf auxiliary(scanner/ssh/ssh_login) > set PASSWORD root123
PASSWORD => root123
msf auxiliary(scanner/ssh/ssh_login) > run
[*] 192.168.31.208:22 - Starting bruteforce
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/ssh/ssh_login) > █
```

```
(kali㉿kali)-[~]
└─$ nmap -sV 192.168.31.208
Starting Nmap 7.95 ( https://nmap.org ) at 2025-08-27 12:36 EDT
Nmap scan report for 192.168.31.208
Host is up (0.0015s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
80/tcp    closed http
443/tcp   closed https
MAC Address: 08:00:27:81:43:E1 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 5.03 seconds
```

```
msf auxiliary(scanner/http/http_version) > run
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.21/lib/recog/fingerprint/regexp_factory.rb
34: warning: nested repeat operator '+' and '?' was replaced with '*' in regular expression
[+] 192.168.31.208:80 Apache/2.4.52 (Ubuntu)
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

5.4 Exploit Assessment:

- Apache exploits were enumerated using search type:exploit name:apache.
- None of the exploits were used actively on the live target, maintaining safe lab conditions.
- Modules like apache_normalize_path_rce and apache_airflow_dag_rce were noted for reporting.

6. Traffic Analysis

6.1 Python Script: traffic_analyzer.py

```
from scapy.all import sniff, IP
import matplotlib.pyplot as plt
from collections import Counter
```

```
packets: list[dict] = []
```

```
def packet_callback(packet):
```

```
    """ Capture IP packets and store metadata """
```

```
    if packet.haslayer(IP):
```

```
        packets.append({'src': packet[IP].src, 'dst': packet[IP].dst, 'size': len(packet)})
```

```
sniff(iface="eth0", prn=packet_callback, count=50)
```

```
print(f"{'Source':<15} {'Destination':<15} {'Size':<5}")
```

```
for pkt in packets:
```

```
print(f"{pkt['src']:<15} {pkt['dst']:<15} {pkt['size']:<5}")
```

```
sizes = [pkt['size'] for pkt in packets]
size_count = Counter(sizes)
plt.bar(size_count.keys(), size_count.values())
plt.title("Packet Size Distribution")
plt.xlabel("Packet Size (Bytes)")
plt.ylabel("Count")
plt.show()
```

Implementation Choices:

- scapy for live packet sniffing.
- matplotlib for visual representation of packet sizes.
- Type hints for Python best practices.
- Capture limited to 50 packets for demonstration.

6.2 Results:

- Captured ICMP traffic between Kali and Ubuntu during ping tests.
- Packet metadata table created.
- Packet size distribution bar chart generated.

```
(kali㉿kali)-[~]
└─$ sudo python3 sniffer.py
[sudo] password for kali:
Ether / ARP is at 58:d9:d5:d6:c0:f8 says 192.168.0.1 / Padding
Ether / IP / UDP 192.168.0.100:51102 > 239.255.255.250:1900 / Raw
Ether / IP / UDP 192.168.0.100:51102 > 239.255.255.250:1900 / Raw
Ether / IP / UDP 192.168.0.100:51102 > 239.255.255.250:1900 / Raw
Ether / IP / UDP 192.168.0.100:51102 > 239.255.255.250:1900 / Raw
Ether / IP / TCP 192.168.0.102:56808 > 142.251.42.36:https PA / Raw
Ether / IP / TCP 192.168.0.102:56808 > 142.251.42.36:https PA / Raw
Ether / IP / TCP 192.168.0.102:56808 > 142.251.42.36:https FA
Ether / IP / TCP 142.251.42.36:https > 192.168.0.102:56808 A
Ether / IP / TCP 142.251.42.36:https > 192.168.0.102:56808 A
```

[illegible]

```
(kali㉿kali)-[~]
$ sudo python3 sniffer.py
[ICMP] 192.168.0.101 → 192.168.0.102 Type=8
[ICMP] 192.168.0.102 → 192.168.0.101 Type=0
[ICMP] 192.168.0.101 → 192.168.0.102 Type=8
[ICMP] 192.168.0.102 → 192.168.0.101 Type=0
[ICMP] 192.168.0.101 → 192.168.0.102 Type=8
[ICMP] 192.168.0.102 → 192.168.0.101 Type=0
[ICMP] 192.168.0.101 → 192.168.0.102 Type=8
[ICMP] 192.168.0.102 → 192.168.0.101 Type=0
[ICMP] 192.168.0.101 → 192.168.0.102 Type=8
[ICMP] 192.168.0.102 → 192.168.0.101 Type=0
[ICMP] 192.168.0.101 → 192.168.0.102 Type=8
[ICMP] 192.168.0.102 → 192.168.0.101 Type=0
```

```
(kali㉿kali)-[~]
$ sudo python3 sniffer.py
[TCP] 192.168.0.102:53638 → 151.101.193.91:443 Flags=PA Payload=46 bytes
[TCP] 151.101.193.91:443 → 192.168.0.102:53638 Flags=A Payload=0 bytes
[TCP] 151.101.193.91:443 → 192.168.0.102:53638 Flags=PA Payload=46 bytes
[TCP] 192.168.0.102:53638 → 151.101.193.91:443 Flags=A Payload=0 bytes
[TCP] 192.168.0.102:53638 → 151.101.193.91:443 Flags=PA Payload=46 bytes
[TCP] 192.168.0.102:53638 → 151.101.193.91:443 Flags=PA Payload=31 bytes
[TCP] 192.168.0.102:53638 → 151.101.193.91:443 Flags=FA Payload=0 bytes
[TCP] 151.101.193.91:443 → 192.168.0.102:53638 Flags=A Payload=0 bytes
[TCP] 151.101.193.91:443 → 192.168.0.102:53638 Flags=A Payload=0 bytes
[TCP] 151.101.193.91:443 → 192.168.0.102:53638 Flags=A Payload=0 bytes
[TCP] 151.101.193.91:443 → 192.168.0.102:53638 Flags=PA Payload=31 bytes
[TCP] 151.101.193.91:443 → 192.168.0.102:53638 Flags=FA Payload=0 bytes
[TCP] 192.168.0.102:53638 → 151.101.193.91:443 Flags=R Payload=0 bytes
[TCP] 192.168.0.102:53638 → 151.101.193.91:443 Flags=R Payload=0 bytes
```

7. Key Learnings & Insights

1. **Penetration Testing Skills:** Learned to perform service enumeration and exploit research using Metasploit modules.
2. **Traffic Analysis:** Gained hands-on experience capturing and analyzing live network packets using Python and Wireshark.
3. **Problem-Solving:** Overcame network misconfiguration challenges by verifying IP addresses and bridging adapters.

8. Appendices & Supporting Evidence

8.1 References:

- Metasploit Framework Documentation: <https://www.metasploit.com/>
- Wireshark User Guide: <https://www.wireshark.org/docs/>
- Scapy Documentation: <https://scapy.readthedocs.io/>